

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



## HỆ ĐIỀU HÀNH

### ĐỒ ÁN 3 – Page Tables

Giảng viên lý thuyết: **Cô Vũ Thị Mỹ Hằng**

Giảng viên thực hành: **Thầy Lê Quốc Hòa**

Nhóm sinh viên thực hiện: **Nhóm 8**

**22120008      Phạm Thiên An**

**22120071      Phan Bá Đức**

**22120215      Nguyễn Thị Mỹ**

**Thành phố Hồ Chí Minh, ngày 29 tháng 12 năm 2024**

## MỤC LỤC

|                                                 |          |
|-------------------------------------------------|----------|
| <b>MỤC LỤC .....</b>                            | <b>2</b> |
| <b>I. THÔNG TIN NHÓM.....</b>                   | <b>3</b> |
| 1. Thông tin thành viên : .....                 | 3        |
| 2. Phân công: .....                             | 3        |
| <b>II. NỘI DUNG.....</b>                        | <b>4</b> |
| 1. Speed up system calls :.....                 | 4        |
| 2. Print a page table : .....                   | 5        |
| 3. Detect which pages have been accessed :..... | 6        |
| <b>TÀI LIỆU THAM KHẢO.....</b>                  | <b>9</b> |

# I. THÔNG TIN NHÓM

## 1. Thông tin thành viên :

| STT | Họ và tên     | MSSV     |
|-----|---------------|----------|
| 1   | Phạm Thiên An | 22120008 |
| 2   | Phan Bá Đức   | 22120071 |
| 3   | Nguyễn Thị Mỹ | 22120215 |

## 2. Phân công:

| STT | Họ và tên     | MSSV     | Phân công                             | Mức độ hoàn thành |
|-----|---------------|----------|---------------------------------------|-------------------|
| 1   | Phạm Thiên An | 22120008 | Speed up system calls                 | 100%              |
| 2   | Phan Bá Đức   | 22120071 | Print a page table                    | 100%              |
| 3   | Nguyễn Thị Mỹ | 22120215 | Detect which pages have been accessed | 100%              |

## II. NỘI DUNG

### 1. Speed up system calls :

#### 1.1. Mô tả yêu cầu :

Triển khai tối ưu hóa để tăng tốc lệnh gọi hệ thống *getpid()* trong Xv6.

#### 1.2. Phương pháp thực hiện :

- Đầu tiên, thêm vào thông tin của một tiến trình một tham số để lưu trữ địa chỉ của trang được chia sẻ.
- Sau đó, trong hàm khởi tạo một tiến trình, phân bổ không gian cho nó và khởi tạo nó để lưu PID của tiến trình hiện tại.
- Tiếp theo, ánh xạ trang chia sẻ vào bảng phân trang trong hàm **proc\_pagetable**.
- Sau đó, giải phóng trang chia sẻ khi giải phóng tiến trình.
- Cuối cùng, hủy ánh xạ các trang trong bảng phân trang trước khi giải phóng bộ nhớ của tiến trình.

#### 1.3. Kết quả thực hiện :

- Khi chạy lệnh *pgtbltest* :

```
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
```

- Khi chạy lệnh *make grade* :

```
== Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
```

**Câu hỏi thêm:** Có thể thực hiện nhanh hơn các lệnh gọi hệ thống xv6 nào khác bằng cách sử dụng trang chia sẻ này? Giải thích cách thực hiện.

Việc sử dụng trang được chia sẻ giữa không gian người dùng và không gian hạt nhân có thể cải thiện hiệu suất cho một số lệnh gọi hệ thống khác trong xv6, đặc biệt là các lệnh gọi có liên quan đến việc lấy dữ liệu từ không gian hạt nhân. Điều này giúp giảm bớt số lần giao thoa vào hạt nhân, vì thay vì thực hiện một cuộc gọi hệ thống tốn kém, tiến trình có thể truy cập trực tiếp dữ liệu trong bộ nhớ chia sẻ. Như *getpid()*, *gettime()*, *getstats()*,...

#### 1.4. Khó khăn (nếu có) :

## 2. Print a page table :

### 2.1. Mô tả yêu cầu :

Viết một hàm in nội dung của bảng trang.

### 2.2. Phương pháp thực hiện :

- Thêm khuôn mẫu hàm `vmprint(pagetable_t)` vào *file defs.h* để có thể gọi sử dụng ở *exec.c*.
- Thêm “`if(p->pid == 1) vmprint(p->pagetable);`” ở trước “`return argc`” để in ra bảng trang của tiến trình đầu tiên (Trong hầu hết các hệ điều hành nhân Linux hoặc nhân giống Unix, tiến trình đầu tiên thường là `init` hoặc tương đương, với `pid = 1`)
- Viết hàm `vmprint helper(pagetable_t pagetable, uint64 deepv)` thực hiện việc in cấu trúc bảng trang (`pagetable`) theo từng cấp bậc (độ sâu `deepv`) trong hệ thống quản lý bộ nhớ
- + Khi hàm được gọi lần đầu (`deepv == 0`), in ra địa chỉ bảng trang gốc
- + Lặp qua tất cả các entry trong bảng trang
  - Kiểm tra tính hợp lệ của entry:
    - Nếu không hợp lệ (`PTE_V` không được set), bỏ qua entry này.
    - Nếu entry hợp lệ, hàm in ra các thông tin:
      - **Cấp độ của bảng trang:** Được biểu thị bằng số lượng ..., tương ứng với `deepv`.
      - **Chỉ mục của entry:** Vị trí trong bảng trang (từ 0 đến 511).
      - **Giá trị PTE:** Giá trị nhị phân của entry, chứa thông tin về cờ và địa chỉ vật lý.
      - **Địa chỉ vật lý:** Tính bằng macro `PTE2PA`, chuyển giá trị trong PTE thành địa chỉ vật lý.
  - Nếu entry hợp lệ và trỏ đến bảng trang cấp thấp hơn, (không ánh xạ trực tiếp đến địa chỉ vật lý, tức là không có cờ `PTE_R`, `PTE_W`, hoặc `PTE_X`):
    - Lấy địa chỉ bảng trang con bằng `PTE2PA`.
    - Gọi lại hàm `vmprint_helper` với bảng trang con và tăng độ sâu (`deepv + 1`).

Hàm tiếp tục được gọi đệ quy với bảng trang con, cho đến khi không còn bảng trang cấp thấp hơn. Mỗi lần gọi, độ sâu (deepv) tăng lên, giúp biểu thị cấp độ trong cây phân cấp.

- Viết hàm chính `vmprint(pagetable_t)`: gọi lại `vmprint()` truyền vào thông tin bảng trang và độ sâu là 0.

### 2.3. Kết quả thực hiện :

- Khi chạy lệnh **make qemu** :

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
page table 0x0000000087f6b000
..0: pte 0x0000000021fd9c01 pa 0x0000000087f67000
....0: pte 0x0000000021fd9801 pa 0x0000000087f66000
.....0: pte 0x0000000021fda01f pa 0x0000000087f68000
.....1: pte 0x0000000021fd941f pa 0x0000000087f65000
.....2: pte 0x0000000021fd9007 pa 0x0000000087f64000
.....3: pte 0x0000000021fd8c17 pa 0x0000000087f63000
..255: pte 0x0000000021fda801 pa 0x0000000087f6a000
....511: pte 0x0000000021fda401 pa 0x0000000087f69000
.....509: pte 0x0000000021fdcc13 pa 0x0000000087f73000
.....510: pte 0x0000000021fdd007 pa 0x0000000087f74000
.....511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
$
```

- Khi chạy **make grade** :

```
$ make qemu-gdb
pte printout: OK (0.9s)
```

### 2.4. Khó khăn (nếu có) :

Ban đầu tách phần if kiểm tra có trở đến trang thấp hơn không ra khỏi if PTE hợp lệ để in thông tin thì mặc dù bảng in ra khá giống với đáp án nhưng chạy `make grade` lại bị sai.

Vì hàm này dựa trên hàm `freewalk` có sẵn nên cũng không quá khó.

## 3. Detect which pages have been accessed :

### 3.1. Mô tả yêu cầu :

Thêm một tính năng vào hệ điều hành xv6 để phát hiện và báo cáo thông tin về các trang bộ nhớ đã được truy cập (đọc hoặc ghi) để tối ưu hóa việc quản lý bộ nhớ cho không gian người dùng bằng cách kiểm tra các bit truy cập trong bảng trang của RISC-V. Bộ dò trang (page walker) của phần cứng RISC-V sẽ đánh dấu các bit này trong PTE (Page Table Entry) mỗi khi xảy ra lỗi TLB (TLB miss).

### 3.2. Phương pháp thực hiện :

#### 1.2.1. Ý tưởng bài toán:

- Ta sẽ thực hiện triển khai hàm **pgaccess()**: Một lệnh gọi hệ thống (system call) để báo cáo các trang bộ nhớ đã được truy cập.
- Hàm này nhận ba tham số:
  - + **Địa chỉ ảo bắt đầu** của trang người dùng đầu tiên cần kiểm tra.
  - + **Số lượng trang** cần kiểm tra.
  - + **Địa chỉ người dùng** trỏ tới một bộ đệm để lưu trữ kết quả vào một bitmask (một cấu trúc dữ liệu sử dụng một bit cho mỗi trang, với trang đầu tiên tương ứng với bit ít quan trọng nhất).
- Ta sẽ sử dụng bit thứ 6 trong PTE (PTE\_A): Một bit dùng để đánh dấu xem trang đó có được truy cập hay không kể từ lần cuối bit đó được xóa.

#### 1.2.2. Các bước thực hiện:

- *Định nghĩa PTE\_A trong file kernel/riscv.h*: Trong PTE nó chính là bit thứ 6. Nhiệm vụ của nó là chỉ ra trang đó đã được truy cập chưa. Nếu nó là 1: tức là được truy cập kể từ lần cuối PTE\_A được ghi là 0 và ngược lại.

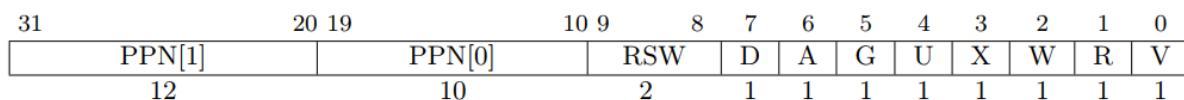


Figure 4.15: Sv32 page table entry.

- *Tiến hành triển khai hàm **sys\_pgaccess()***:
  - + Bước 1: Lấy tham số từ không gian người dùng: Ta sẽ dùng **argint()** để truy xuất tham số **num\_pages** (số lượng trang) và **argaddr()** để lấy địa chỉ ảo bắt đầu và địa chỉ người dùng.
  - + Bước 2: Ta sẽ giới hạn số trang sẽ được kiểm tra, nếu muốn ta cũng có thể thay đổi.
  - + Bước 3: Ta sẽ tính địa chỉ ảo của trang cần kiểm tra và sử dụng hàm **walk()** để tìm PTE tương ứng trong bảng trang. Nếu không tìm thấy trả về lỗi -1.

+ Bước 4: Kiểm tra trang hợp lệ qua PTE\_V và trang đã được truy cập qua PTE\_A, nếu thỏa 2 điều kiện trên ta sẽ đánh dấu trang đó lại (mỗi bit trong biến **buf** đại diện cho trạng thái của mỗi trang với 1 nghĩa là trang đã được truy cập và ngược lại), đồng thời ta sẽ cho bit PTE\_A về 0. Nếu không làm thế, ta sẽ không thể xác định xem trang (trước khi được kiểm tra) có được truy cập không trong các lần kiểm tra tiếp theo. Cứ tiếp tục thực hiện bước 3 và 4 cho đến khi duyệt xong hết các trang.

+ Bước 5: Lưu thông tin các trang đã được truy cập được lưu vào biến **buf** khi này vào trong **buffer\_addr** trong không gian người dùng thông qua hàm **copyout()**.

### 3.3. Kết quả thực hiện :

- Khi chạy lệnh **pgtbltest** :

```
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
pgaccess_test starting
pgaccess_test: OK
pgtbltest: all tests succeeded
```

- Khi chạy **make grade** :

```
== Test   pgtbltest: pgaccess ==
pgtbltest: pgaccess: OK
```

### 3.4. Khó khăn (nếu có) :

- Lúc đầu không biết sử dụng đúng cách file test nên nghĩ mình làm sai .



# **TÀI LIỆU THAM KHẢO**

**Lab: page tables** | <https://pdos.csail.mit.edu/6.1810/2023/labs/pgtbl.html>